

**Examen POO pour le réseau**

le 1 Juin 2022

RSI M1

Nom**Prénom****Durée 1h30****Des questions peuvent avoir plusieurs réponses****Le Questions sont sur 12 points, Le problème sur 8 points****Question 1 :**

Soit l'instruction suivante, dessinez le diagramme de classe correspondant :

```
DataSet ds=new Shuffle(new Mapper( new Reducer(
new RDD()))).
```

fournisseur lorsque des impertinences apparaissent pendant le service, quel DP est le mieux approprié :

- a. Adapter
- b. Decorator
- c. Observer
- d. Proxy

Question 2:

Dans RMI, La sérialisation est un processus

- a. qui envoie les données depuis et vers un port série
- b. qui impose la simplification des objets complexes avant leur envoi
- c. qui nécessite l'implémentation de l'interface Listner
- d. qui implique l'envoi des donnée sous forme d'une suite de bits

Question 3 :

Dans RMI coté Client, on doit faire un downcasting sur la méthode Lookup() parce que :

- a. L'interface partagée entre le client et le serveur est sérialisable
- b. Les objets dans RMI registry sont du type Object
- c. Le client ne peut pas accéder à la classe définissant l'objet partagé
- d. L'objet enregistré avec RMI registry est du type de l'interface

Question 4 :

Les fournisseurs de Cloud doivent concevoir des applications de Monitoring pour assurer une bonne qualité de service à leurs Clients et pour avertir le

Question 5 : Dans RMI où se trouve l'objet distant

- a. Impérativement Dans la machine du serveur
- b. Dans un annuaire appelé rmi registry
- c. Dans une machine distante au client
- d. Là où est localisée la classe d'implémentation du serveur

Question 6 : Corba et Rmi ont des points en commun :

- a. Le client et le serveur doivent implémenter la même interface
- b. Ils sont spécifiques au langage Java
- c. Ils utilisent des annuaires où le client peut chercher l'objet distant
- d. Le canal de communication entre le client et le serveur doit implémenter POA

Question 7: Dans l'exercice 3 de la série 1, pourquoi on n'a pas implémenté la méthode remove() de la classe Iterator ?

- a. Parce qu'elle existe déjà dans la classe Enumeration
- b. Parce qu'avec l'héritage on ne redéfinit pas les méthodes existantes
- c. Parce qu'on est entrain d'adapter enumeration à Iterator
- d. Parce que la méthode remove se trouve uniquement dans Iterator



Question 8 : Que changeriez-vous dans le code de la classe démo, de l'exercice 4, série 4 pour que soit affiché le résultat suivant :

- a) Weather customer 2 just found out the temperature is: 40
- b) Weather customer 1 just found out the temperature is: 40
- c) Weather customer 3 just found out the temperature is: 40
- d) Weather station setting temperature to 20
- e) Weather customer 2 just found out the temperature is: 20
- f) Weather customer 1 just found out the temperature is: 20
- g)

a) rajouter une classe WeatherCustomer3 et écrire le code: WeatherStation weatherStation = new WeatherStation(20); WeatherCustomer1 wc1 = new WeatherCustomer1(); WeatherCustomer2 wc2 = new WeatherCustomer2(); weatherStation.addObserver(wc1); weatherStation.addObserver(wc2); weatherStation.addObserver(wc3); weatherStation.setTemperature(40); weatherStation.removeObserver(wc3); weatherStation.setTemperature(40);	b) rajouter une classe WeatherCustomer 3 et écrire le code: WeatherStation weatherStation = new WeatherStation(40); WeatherCustomer1 wc1 = new WeatherCustomer1(); WeatherCustomer2 wc2 = new WeatherCustomer2(); WeatherCustomer2 wc3 = new WeatherCustomer3(); weatherStation.addObserver(wc1); weatherStation.addObserver(wc2); weatherStation.addObserver(wc3); weatherStation.setTemperature(20);	c) rajouter une classe WeatherCustomer 3 et écrire le code: WeatherStation weatherStation = new WeatherStation(30); WeatherCustomer1 wc1 = new WeatherCustomer1(); WeatherCustomer2 wc2 = new WeatherCustomer2(); WeatherCustomer2 wc3 = new WeatherCustomer3(); weatherStation.addObserver(wc1); weatherStation.addObserver(wc2); weatherStation.addObserver(wc3); weatherStation.setTemperature(40); weatherStation.removeObserver(wc3); weatherStation.setTemperature(20);
--	---	--

Problème:

Les attaques Multi-étapes ont pour but de nuire de manière persistante leur victimes en suivant plusieurs étapes : probing attack, sadmind exploit, DDos,..quel est le design pattern qu'un attaquant utilisera pour aboutir à sa victime de manière séquentielle ? Donner une implémentation possible. Pour contrer ce type d'attaques, les solutions proposées s'appuient sur les graphes d'attaques, de telle sorte que chaque étape est représentée par un nœud, chaque noeud va subir la même analyse, concernant sa relation temporelle et d'adresse IP avec celui qui le précède. Quel design pattern est approprié pour cette étape d'analyse, donner une esquisse de code ?